

Assignment 7.3

M.koushik reddy

2103A52153

Task 1

```
# Corrected function definition
def add(a, b):
    return a + b

# Test cases with basic error handling
try:
    print("Test 1:", add(2, 3))      # Expected: 5
    print("Test 2:", add(-1, 4))    # Expected: 3
    print("Test 3:", add(0, 0))     # Expected: 0
except TypeError:
    print("Error: Please provide numeric values.")
```

Output

```
task 1.py"
Test 1: 5
Test 2: 3
Test 3: 0
PS C:\Users\koushik_reddy\Desktop\python practice>
```

Task 2

```
def count_down(n):
```

```
while n >= 0:
    print(n)
    n -= 1    # Correct decrement

# Test
count_down(5)
```

Output

```
PS C:\Users\koushik reddy\Desktop\python practice> &
task 1.py"
5
4
3
2
1
0
```

Task 3

```
# Function with division
def divide(a, b):
    return a / b

# Safe execution with error handling
try:
    print(divide(10, 0))
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Output

Error: Cannot divide by zero.

Explanation:

The program originally crashed because of a **division by zero runtime error**. When the function `divide(10, 0)` is executed, Python raises a `ZeroDivisionError` since dividing any number by zero is not allowed.

To fix this issue, a **try-except block** is used. The risky code is placed inside the try block, and the except `ZeroDivisionError` block handles the error if it occurs. Instead of stopping the program, a user-friendly message is displayed.

This approach improves program reliability by preventing crashes and allowing safe execution even when invalid input is provided

Task 4

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

# Test
r = Rectangle(10, 5)
print("Length:", r.length)
print("Width:", r.width)
```

Output

Length: 10

Width: 5

Task 5

```
numbers = [1, 2, 3]

try:
    print(numbers[5])
except IndexError:
    print("Error: Index out of range.")
```

Output

Error: Index out of range.

Explanation:

The program crashes because it tries to access an index that does not exist in the list. The list `numbers = [1, 2, 3]` has only three elements, so the valid indexes are 0, 1, and 2.

Accessing `numbers[5]` causes an `IndexError`.

To fix this, a try-except block is used to catch the error.

Instead of the program stopping, it prints an error message.

This makes the program safer and prevents it from crashing when an invalid index is used.

